Computer Practical

Saturn's Hexagon

Parallel shallow-water modelling on a rotating sphere

Paul Connolly • September 2026

1 Overview

In this short practical you will use a shallow-water model on a rotating sphere to investigate Saturn's polar jet and the development of polygonal structure. The model is written in Fortran and uses MPI so that different parts of the domain can be calculated in parallel.

This practical builds directly on the earlier shallow-water practical. The main differences are that the domain is spherical, so its circumference changes with latitude, and the model is run on several processors at once.

Saturn's Hexagon is observed near 77°N. Cloud motions can be used to estimate atmospheric winds, but defining the rotation rate of a planet without a solid surface is less straightforward. A commonly used estimate comes from periodic radio emissions associated with Saturn's interior, giving a rotation period of roughly 10.5 hours.

By the end of the practical you should be able to:

- run a simple MPI model using several processors;
- examine how the simulated polar flow changes as the jet speed is increased;
- identify where cyclonic and anticyclonic structures develop around the jet.

2 Download and compile the model

Use the same SSH/SFTP workflow as in *Tools of the Trade*. In your SSH window, download the repository if you have not already done so:

```
git clone https://github.com/EnvModelling/shallow-water-model-on-sphere
cd shallow-water-model-on-sphere
```

Compile the Fortran model with:

```
make
```

The executable produced by the build is `main.exe`.

3 Configure the experiment

The model settings are in `namelist.in`. Open the file with line numbers displayed:

```
nano -l namelist.in
```

The main parameter used in this practical is `nm1%u_jet`, at approximately line 11. It sets the maximum speed of the idealised jet in m s^{-1}. If the line number has changed, use `Ctrl-W` in `nano` and search for `u_jet`.

Save changes with `Ctrl-X`, then press `Y` and `Enter`.

4 Run the model

Run the model from the repository root. On the teaching server, use 12 MPI tasks:

```
./run.sh 12
```

The script limits the maximum number of tasks that can be requested. If the server is busy, Slurm will leave the job in the queue until processors become available. The same `run.sh` command can also run on a system without Slurm, using `mpiexec` instead.

When the run completes successfully, the model output is placed at:

```
/tmp/YOUR_USERNAME/output.nc
```

A later simulation will replace this file, so generate and save the plots you want before starting the next run.

5 Plot and download the results

To make a still image of the final state, type:

```
python3 python/height_and_streamlines.py
```

This creates:

```
/tmp/YOUR_USERNAME/frame.png
```

To make an animation, type:

```
python3 python/animate_output.py
```

This creates:

```
/tmp/YOUR_USERNAME/animation.gif
```

Use your SFTP window to download the result. Give the local copy a descriptive name so that later simulations do not overwrite it. For example:

```
get /tmp/YOUR_USERNAME/animation.gif saturn_60.gif
```

For a still image, the same idea applies:

```
get /tmp/YOUR_USERNAME/frame.png saturn_60.png
```

6 Experiments

Change only `nm1%u_jet` between runs. Use the following jet speeds:

Run	u_jet
1	60 m s^{-1}
2	90 m s^{-1}
3	120 m s^{-1}
4	200 m s^{-1}
5	420 m s^{-1}

For each simulation:

1. edit `namelist.in`;
2. run `./run.sh 12`;
3. generate the still image or animation;
4. download it using a filename that records the jet speed.

Exercise. Where do the cyclones and anticyclones form relative to the jet?

Exercise. What happens to the simulated flow as the jet speed is increased?

Note. Compare the simulations with one another rather than treating each run in isolation. The aim is to identify the systematic change caused by increasing `u_jet`.